

Documentation V-Check

V-Check – Application mobile de supervision

Présentation du projet

V-Check est une application mobile développée avec **React Native** et **Expo** permettant aux utilisateurs de consulter l'état d'un environnement de serveurs virtualisés. L'authentification est gérée par **Supabase**, et les métriques système sont remontées depuis un serveur **Proxmox** monitoré par **Prometheus**.

Le but de ce projet est de fournir une interface mobile légère, moderne et facile à utiliser pour surveiller les performances et la santé de machines virtuelles.

Technologies utilisées

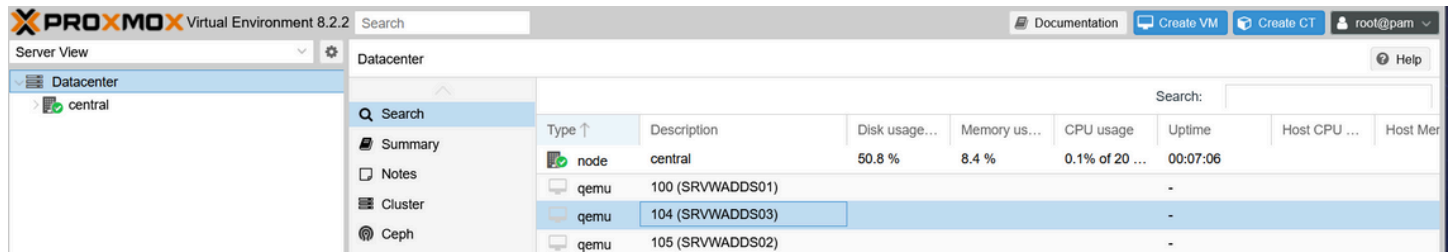
- **React Native + Expo** : Pour le développement mobile cross-platform.
- **Supabase** : Authentification simple via une base de données PostgreSQL.
- **Prometheus** : Collecte des métriques système.
- **Proxmox VE (8.2.2)** : Virtualisation des machines.
- **Grafana** (*mentionné si tu le connectes ensuite*) : Visualisation des données Prometheus.

Structure de l'application

```
|—— app/
|   |—— index.js          // Page de login
|   |—— (tabs)/App.js     // Page principale après connexion
|—— lib/
```

- └─── supabase.js // Configuration de Supabase
- └─── assets/ // Images, icônes
- └─── app.json // Configuration Expo
- └─── package.json // Dépendances

Aperçu du serveur Proxmox:



Fichier App.js:

```

1 import React, { useEffect, useState } from "react";
2 import { View, Text, StyleSheet, TouchableOpacity, Dimensions } from "react-native";
3 import axios from "axios";
4 import { ProgressBar } from "react-native-paper";
5 import { MaterialIcons } from "@expo/vector-icons";
6
7
8 const PROMETHEUS_URL = "http://10.0.0.25:9090/api/v1/query";
9 export default function SystemUsage() {
10   const [selectedVM, setSelectedVM] = useState("10.0.0.27");
11   const [cpuUsage, setCpuUsage] = useState(null);
12   const [ramUsage, setRamUsage] = useState(null);
13   const [storageUsage, setStorageUsage] = useState(null);
14   const [networkUsage, setNetworkUsage] = useState(null);
15   const [isVMOnline, setIsVMOnline] = useState(true);
16   const [dnsStatus, setDnsStatus] = useState(null);
17   useEffect(() => {
18     fetchMetrics();
19     const interval = setInterval(fetchMetrics, 5000);
20     return () => clearInterval(interval);
21   }, [selectedVM]);
22   const fetchMetrics = async () => {
23     try {
24       console.log("🔄 Récupération des métriques pour la VM ${selectedVM}...");
25       // Vérification si la VM est UP
26       const upQuery = `up{instance="${selectedVM}:9182"}`;
27       const upResponse = await axios.get(PROMETHEUS_URL, {
28         params: { query: upQuery },
29       });
30       const isOnline = upResponse.data.data.result.length > 0 &&
31         parseFloat(upResponse.data.data.result[0].value[1]) === 1;
32       setIsVMOnline(isOnline);
33       if (!isOnline) {
34         console.log("🔴 VM ${selectedVM} OFFLINE");
35         setCpuUsage(null);
36         setRamUsage(null);
37         setStorageUsage(null);
38         setNetworkUsage(null);
39         setDnsStatus(null);
40         return;
41       }
42       const queries = {
43         cpu: `100 - (avg by(instance) (rate(windows_cpu_time_total{mode="idle", instance="${selectedVM}:9182"}[5m]))) * 100`,
44         ram: `100 * (1 - (windows_memory_available_bytes{instance="${selectedVM}:9182"} / windows_cs_physical_memory_bytes{instance="${selectedVM}:9182"}))`,
45         storage: `100 * (1 - (windows_logical_disk_free_bytes{instance="${selectedVM}:9182", volume="C:"} / windows_logical_disk_size_bytes{instance="${selectedVM}:9182", volume="C:"}))`,
46         network: `rate(windows_net_bytes_received_total{instance="${selectedVM}:9182"}[5m]) + rate(windows_net_bytes_sent_total{instance="${selectedVM}:9182"}[5m])`,
47         dns: `windows_service_state{instance="${selectedVM}:9182", name="DNS", state="running"}`;
48       };
49       const fetchMetric = async (query) => {
50         const response = await axios.get(PROMETHEUS_URL, {

```

```

51     params: { query },
52     paramsSerializer: (params) =>
53       Object.keys(params)
54         .map((key) => `${encodeURIComponent(key)}=${encodeURIComponent(params[key])}`)
55         .join("&"),
56   ));
57   return response.data.data.result.length > 0
58     ? parseFloat(response.data.data.result[0].value[1])
59     : null;
60 };
61 const [cpu, ram, storage, network, dns] = await Promise.all([
62   fetchMetric(queries.cpu),
63   fetchMetric(queries.ram),
64   fetchMetric(queries.storage),
65   fetchMetric(queries.network),
66   fetchMetric(queries.dns),
67 ]);
68 setCpuUsage(cpu);
69 setRamUsage(ram);
70 setStorageUsage(storage);
71 setNetworkUsage(network ? network / 1048576 : 0.0001);
72 setDnsStatus(dns === 1 ? "Actif" : "Inactif");
73 } catch (error) {
74   console.error("❌ Erreur Prometheus API:", error.message);
75   setIsVMOnline(false);
76   setCpuUsage(null);
77   setRamUsage(null);
78   setStorageUsage(null);
79   setNetworkUsage(null);
80   setDnsStatus(null);
81 }
82 };
83 return (
84   <View style={styles.container}>
85     <Text style={styles.title}>🖥️ Surveillance du Système</Text>
86     <View style={styles.buttonContainer}>
87       <TouchableOpacity style={[styles.button, selectedVM === "10.0.0.27" && styles.buttonActive]} onPress={() => setSelectedVM("10.0.0.27")}>
88         <Text style={styles.buttonText}>SRVWADDS02</Text>
89       </TouchableOpacity>
90       <TouchableOpacity style={[styles.button, selectedVM === "10.0.0.48" && styles.buttonActive]} onPress={() => setSelectedVM("10.0.0.48")}>
91         <Text style={styles.buttonText}>SRVWADDS03</Text>
92       </TouchableOpacity>
93       <TouchableOpacity style={[styles.button, selectedVM === "10.0.0.55" && styles.buttonActive]} onPress={() => setSelectedVM("10.0.0.55")}>
94         <Text style={styles.buttonText}>SRVWADDS01</Text>
95       </TouchableOpacity>

```

```

95     </TouchableOpacity>
96 </View>
97 {!isVMOnline && (
98   <Text style={styles.offlineText}>🚫 La VM est hors ligne</Text>
99 )}
100 [[
101   { label: "CPU", value: cpuUsage, color: "#ff3e3e", icon: "memory" },
102   { label: "RAM", value: ramUsage, color: "#3e85ff", icon: "storage" },
103   { label: "Stockage", value: storageUsage, color: "#ff9800", icon: "sd-storage" },
104   { label: "Réseau", value: networkUsage, color: "#00e5ff", icon: "wifi", unit: "Mo/s", decimals: 4 },
105   { label: "DNS", value: dnsStatus, color: "#4caf50", icon: "dns", unit: "", decimals: 0 },
106 ].map(({ label, value, color, icon, unit = "%", decimals = 2 }, index) => (
107   <View key={index} style={styles.metricContainer}>
108     <View style={styles.iconContainer}>
109       <MaterialIcons name={icon} size={30} color={color} />
110       <Text style={styles.metricTitle}>{label}</Text>
111     </View>
112     {value !== null ? (
113       <>
114         <Text style={styles.percentage}>
115           {typeof value === "number" ? value.toFixed(decimals) : value} {unit}
116         </Text>
117         {label !== "DNS" && <ProgressBar progress={Math.min(value / 100, 1)} color={color} style={styles.progressBar} />}
118       </>
119     ) : (
120       <Text style={styles.loading}>Aucune donnée disponible</Text>
121     )}
122   </View>
123 ))}
124 </View>
125 );
126 }
127 const styles = StyleSheet.create({
128   container: {
129     flex: 1,
130     padding: "2%", // Réduction du padding global
131     backgroundColor: "#1e1e1e",
132     alignItems: "center",
133     justifyContent: "center"
134   },
135   title: {
136     fontSize: Dimensions.get('window').width * 0.05, // Réduction de la taille de la police
137     fontWeight: "bold",
138     color: "ffffff",
139     marginBottom: 15 // Réduction de la marge inférieure
140   },
141   metricContainer: {

```

```

142     width: "85%", // Réduction de la largeur
143     marginBottom: 15, // Réduction de la marge inférieure
144     padding: "3%", // Réduction du padding
145     borderRadius: 10,
146     backgroundColor: "#292929",
147     alignItems: "center",
148     shadowColor: "#000",
149     shadowOpacity: 0.3,
150     shadowRadius: 5,
151     elevation: 5
152   },
153   iconContainer: {
154     flexDirection: "row",
155     alignItems: "center",
156     marginBottom: 6 // Réduction de la marge inférieure
157   },
158   metricTitle: {
159     fontSize: Dimensions.get('window').width * 0.045, // Réduction de la taille de la police
160     fontWeight: "bold",
161     color: "ffffff",
162     marginLeft: 8 // Réduction de la marge gauche
163   },
164   percentage: {
165     fontSize: Dimensions.get('window').width * 0.05, // Réduction de la taille de la police
166     fontWeight: "bold",
167     color: "ffffff",
168     marginBottom: 6 // Réduction de la marge inférieure
169   },
170   progressBar: {
171     width: "100%",
172     height: 10, // Réduction de la hauteur
173     borderRadius: 5,
174     backgroundColor: "#404040"
175   },
176   loading: {
177     fontSize: Dimensions.get('window').width * 0.035, // Réduction de la taille de la police
178     color: "gray"
179   },
180   buttonContainer: {
181     width: "85%", // Réduction de la largeur
182     flexDirection: "column",
183     marginBottom: 15 // Réduction de la marge inférieure
184   },
185   button: {
186     backgroundColor: "#333",
187     paddingVertical: 10, // Réduction du padding vertical
188     paddingHorizontal: 15, // Réduction du padding horizontal

```

```

188     paddingHorizontal: 15, // Réduction du padding horizontal
189     borderRadius: 8,
190     marginVertical: 4, // Réduction de la marge verticale
191     alignItems: "center"
192   },
193   buttonActive: {
194     backgroundColor: "#007bff"
195   },
196   buttonText: {
197     color: "#fff",
198     fontWeight: "bold",
199     fontSize: Dimensions.get('window').width * 0.035 // Réduction de la taille de la police
200   },
201   offlineText: {
202     fontSize: Dimensions.get('window').width * 0.035, // Réduction de la taille de la police
203     color: "#ff3e3e",
204     marginBottom: 8 // Réduction de la marge inférieure
205   }
206 });

```

Fichier index.js:

```

1  import { useState } from 'react'
2  import { View, TextInput, Button, Text, StyleSheet, Alert } from 'react-native'
3  import { supabase } from '../lib/supabase'
4  import { useRouter } from 'expo-router'
5
6  export default function Index() {
7    const router = useRouter()
8    const [nomUtilisateur, setNomUtilisateur] = useState('')
9    const [motDePasse, setMotDePasse] = useState('')
10   const [loading, setLoading] = useState(false)
11
12   const handleLogin = async () => {
13     setLoading(true)
14     const { data, error } = await supabase
15       .from('login')
16       .select('*')
17       .eq('nom_utilisateur', nomUtilisateur.trim())
18       .eq('mot_de_passe', motDePasse.trim())
19       .single()
20
21     console.log("Nom utilisateur:", nomUtilisateur)
22     console.log("Mot de passe:", motDePasse)
23     console.log("Données Supabase:", data)
24     console.log("Erreur Supabase:", error)
25
26
27     setLoading(false)
28
29     if (error || !data) {
30       Alert.alert('Échec de la connexion', 'Nom d'utilisateur ou mot de passe incorrect.')
31     } else {
32       router.replace('/(tabs)/App') // redirection après login
33     }
34   }
35
36   return (
37     <View style={styles.container}>
38       <Text style={styles.title}>Connexion</Text>
39       <TextInput
40         placeholder="Nom d'utilisateur"

```

```

42     value={nomUtilisateur}
43     onChangeText={setNomUtilisateur}
44     style={styles.input}
45   />
46   <TextInput
47     placeholder="Mot de passe"
48     value={motDePasse}
49     onChangeText={setMotDePasse}
50     secureTextEntry
51     style={styles.input}
52   />
53   <Button title={loading ? 'Connexion...' : 'Se connecter'} onPress={handleLogin} />
54 </View>
55 )
56 }
57
58 const styles = StyleSheet.create({
59   container: {
60     flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20,
61   },
62   title: {
63     fontSize: 24, fontWeight: 'bold', marginBottom: 20,
64   },
65   input: {
66     width: '100%', height: 40, borderColor: 'gray', borderWidth: 1,
67     marginBottom: 12, paddingHorizontal: 10, borderRadius: 5
68   }
69 })
70

```